

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЁТ

По лабораторной работе №5
По курсу «Разработка кроссплатформенных приложений»
на тему «Работа с файлами»

Выполнили
студенты группы 23ВВИ2:

Родионов А. А.

Пырков Д. А.

Приняли:
Юрова О.В.
Карамышева Н.С.

Пенза 2026

Цель работы: научиться создавать многопоточные приложения с использованием стандартных средств языка Java.

Лабораторное задание: модифицировать приложение из предыдущей лабораторной работы, реализовав вычисление определенного интеграла в нескольких дополнительных потоках (число потоков определяется номером варианта), снимая нагрузку с основного потока и предотвращая "подвисание" графического интерфейса. Варианты с номерами до 5 включительно реализуют многопоточность путем наследования от класса Thread, остальные реализуют интерфейс Runnable.

Ход работы

Описание работы программы

В среде NetBeans был создан класс `IntegralThread` наследуется от стандартного класса `Thread`. Каждый экземпляр этого класса представляет собой отдельный поток выполнения, ответственный за вычисление интеграла на одном подотрезке. При создании объекта в конструктор передаются границы подотрезка `from` и `to`, а также шаг интегрирования `step`. Эти значения сохраняются в полях объекта и используются при выполнении потока.

Основная логика вычисления реализована в переопределённом методе `run()`. Именно этот метод вызывается средой исполнения, когда поток начинает работу. Внутри `run()` выполняется классический алгоритм метода трапеций: на каждом шаге вычисляются значения функции $\sin(x)$ на левой и правой границах текущего отрезка, после чего площадь трапеции прибавляется к накопленному частичному результату `partialResult`. Последний шаг может оказаться неполным, если длина подотрезка не делится нацело на шаг. В этом случае вычисляется трапеция с укороченным основанием, равным расстоянию от последней пройденной точки до правой границы подотрезка.

После завершения работы метода `run()` частичный результат доступен через геттер `getPartialResult()`. Важно понимать, что вызывать этот метод имеет

смысл только после того, как поток гарантированно завершился, иначе возвращённое значение может оказаться неполным.

Метод `calculateResult` класса `RecIntegral`

Метод `calculateResult()` является точкой входа для запуска многопоточного вычисления. Он вызывается из основного потока приложения, но сам по себе не выполняет численное интегрирование. Вместо этого он делегирует вычислительную работу дополнительным потокам.

На первом этапе метод вычисляет длину подотрезка, приходящегося на каждый поток. Для этого длина всего отрезка интегрирования `[lowerLimit, upperLimit]` делится на значение константы `THREAD_COUNT`. Затем в цикле создаются экземпляры класса `IntegralThread`, каждому из которых назначаются свои границы. Для последнего потока правая граница явно устанавливается равной `upperLimit`, чтобы избежать накопления ошибки округления при последовательном сложении длин подотрезков.

Сразу после создания каждого потока вызывается метод `start()`. Этот метод унаследован от класса `Thread` и является стандартным механизмом запуска потока в Java. Вызов `start()` не приводит к немедленному выполнению метода `run()` в текущем потоке. Вместо этого JVM создаёт новый системный поток, в котором будет асинхронно вызван метод `run()`. Таким образом, после завершения цикла создания и запуска все потоки работают параллельно друг с другом и параллельно основному потоку.

На втором этапе основной поток должен дождаться завершения всех вычислительных потоков, прежде чем собирать результат. Для этого используется метод `join()`, вызываемый для каждого потока в цикле. Вызов `join()` блокирует текущий (основной) поток до тех пор, пока целевой поток не завершит выполнение своего метода `run()`. Если поток уже завершился к моменту вызова `join()`, блокировки не происходит и управление возвращается немедленно. Метод `join()` может выбросить `InterruptedException`, если основной

поток будет прерван во время ожидания. В этом случае исключение оборачивается в `RuntimeException` и пробрасывается выше.

После того как все потоки завершились, основной поток последовательно извлекает частичные результаты через `getPartialResult()` и суммирует их. Полученная сумма сохраняется в поле `result` объекта `RecIntegral`. На этом вычисление считается завершённым.

Потокобезопасность

Потоки в данной реализации не разделяют между собой изменяемые данные. Каждый поток работает исключительно со своим подотрезком и записывает результат в своё собственное поле `partialResult`. Основной поток читает эти поля только после вызова `join()`, который обеспечивает отношение «happens-before» между завершением дочернего потока и последующим чтением данных в основном потоке. Благодаря этому никакие дополнительные средства синхронизации, такие как `synchronized`, `volatile` или блокировки, не требуются.

Выигрыш от параллелизма

Основной поток полностью освобождён от вычислительной нагрузки. Он лишь создаёт потоки, ожидает их завершения и складывает два числа. Вся ресурсоёмкая работа выполняется в дополнительных потоках, каждый из которых обрабатывает ровно половину отрезка интегрирования. На многоядерном процессоре оба потока могут выполняться одновременно на разных ядрах, что позволяет сократить общее время вычисления приблизительно вдвое по сравнению с последовательной версией.

Тестирование программы

Проведём тестирование программы.

Для проверки корректности вычислений выполним расчёт трёх интегралов в среде PTC MathCad.

$$\int_{0.1}^{99} \sin(x) dx = 0.9551832849$$

$$\int_{50}^{199} \sin(x) dx = 1.436591741$$

$$\int_{25}^{500} \sin(x) dx = 1.8750520853$$

Далее вычислим те же самые интегралы при помощи разработанной программы.

Integral calculator

Таблица
Коллекция
Файлы

Верхний предел
 Нижний предел
 Шаг

Добавить
Заполнить
Сохранить (T)

Удалить
Очистить
Загрузить (T)

Рассчитать
Sin(x)
Сохранить (Bin)

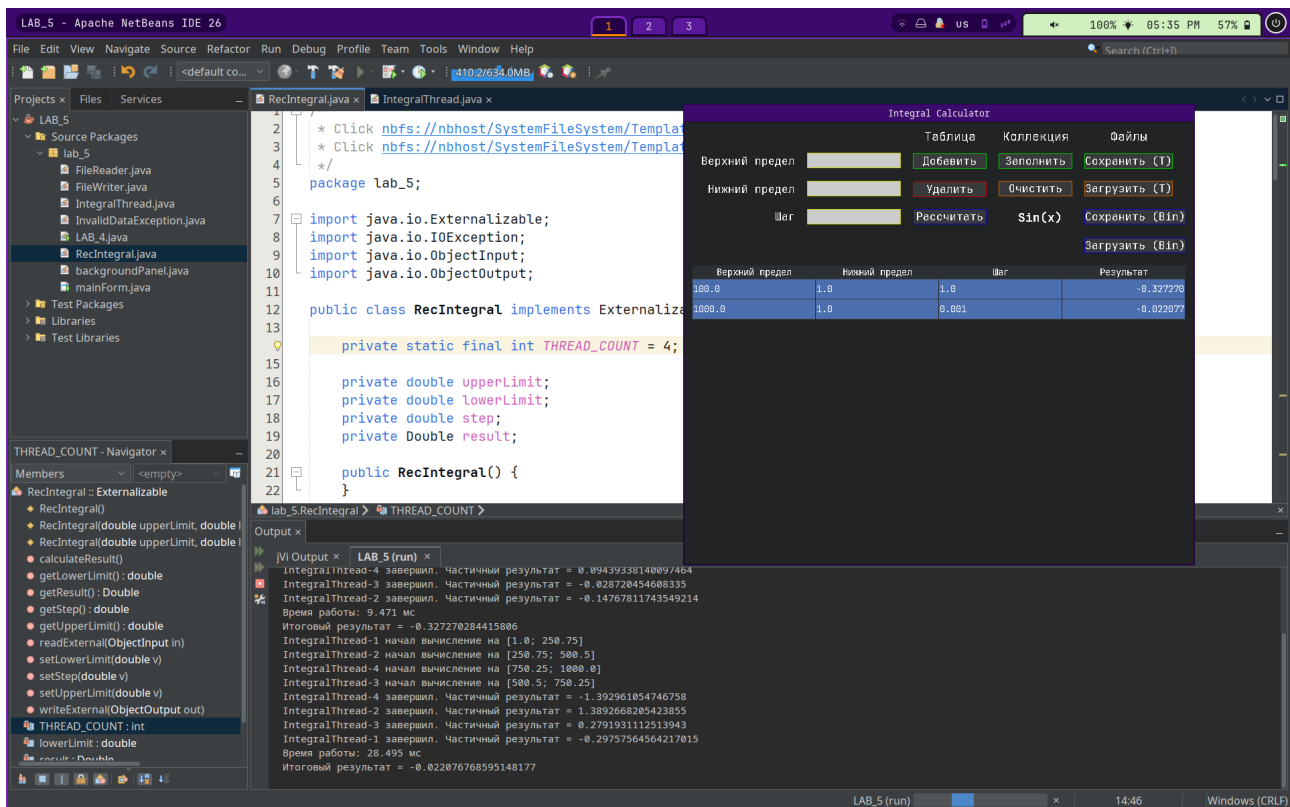
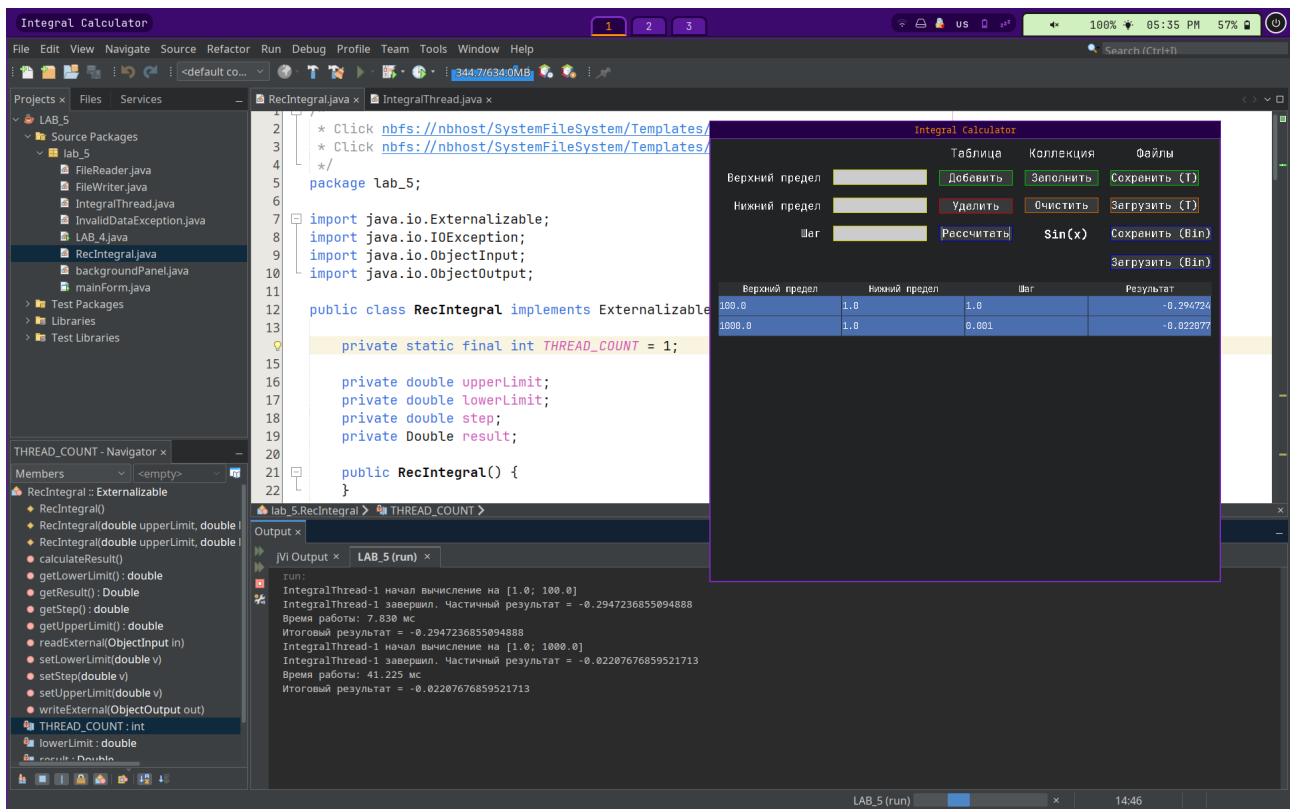
Загрузить (Bin)

Верхний предел	Нижний предел	Шаг	Результат
500.0	25.0	0.001	1.875052
199.0	50.0	0.001	1.436592
99.0	0.1	0.001	0.955183

Результаты работы программ совпадают. В программе также есть вывод в терминал для отслеживания результатов каждого потока.

```
IntegralThread-1 начал вычисление на [25.0; 262.5]
IntegralThread-2 начал вычисление на [262.5; 500.0]
IntegralThread-2 завершил. Частичный результат = 1.059939641755507
IntegralThread-1 завершил. Частичный результат = 0.815112287303831
Итоговый результат = 1.875051929059338
IntegralThread-1 начал вычисление на [50.0; 124.5]
IntegralThread-2 начал вычисление на [124.5; 199.0]
IntegralThread-1 завершил. Частичный результат = 0.5690269904172706
IntegralThread-2 завершил. Частичный результат = 0.8675646308759607
Итоговый результат = 1.4365916212932313
IntegralThread-1 начал вычисление на [0.1; 49.550000000000004]
IntegralThread-2 начал вычисление на [49.550000000000004; 99.0]
IntegralThread-1 завершил. Частичный результат = 0.24022729941640392
IntegralThread-2 завершил. Частичный результат = 0.7149559058721665
Итоговый результат = 0.9551832052885705
IntegralThread-1 начал вычисление на [25.0; 262.5]
IntegralThread-2 начал вычисление на [262.5; 500.0]
IntegralThread-1 завершил. Частичный результат = 0.815112287303831
IntegralThread-2 завершил. Частичный результат = 1.059939641755507
Итоговый результат = 1.875051929059338
IntegralThread-1 начал вычисление на [50.0; 124.5]
IntegralThread-2 начал вычисление на [124.5; 199.0]
IntegralThread-1 завершил. Частичный результат = 0.5690269904172706
IntegralThread-2 завершил. Частичный результат = 0.8675646308759607
Итоговый результат = 1.4365916212932313
IntegralThread-1 начал вычисление на [0.1; 49.550000000000004]
IntegralThread-2 начал вычисление на [49.550000000000004; 99.0]
IntegralThread-1 завершил. Частичный результат = 0.24022729941640392
IntegralThread-2 завершил. Частичный результат = 0.7149559058721665
Итоговый результат = 0.9551832052885705
```

Далее произведём замер времени работы программы в зависимости от количества потоков. В данном случае сравнение будет проводится между одним и четырьмя потоками.



Как видно из результатов тестирования, при расчёте с большим пределом и маленьким шагом результат лучше на четырёх потоках, а когда предел маленький и шаг большой, то лучше один поток.

Тестирование завершено, программа работает корректно.

Вывод: в ходе выполнения лабораторной работы были получены навыки разработки многопоточных приложений с использованием стандартных средств языка Java..